

ChronoNexus

Master the Ripple, Conquer the Deadline.

PROJECT REPORT



Name-Dhruv Shrivastava
Registration No.-25BA111236
Course- Programming in JAVA
Slot - B11 + B12
Faculty - Dr. Dheresh Soni

CHRONONEXUS

- **INTRODUCTION**

ChronoNexus is an extremely sophisticated CLI-based project and task management scheduler written in java programming language. Apart from the traditional to-do list, ChronoNexus uses Directed Acyclic Graphs (DAG) for handling task dependency, computing critical path, and simulating the ripple effect of delays on the project timeline. This tool is meant for developers and students who want a lightweight mathematical scheduler without using enterprise level GUI application.

- **PROBLEM STATEMENT**

Management of the complex deliverables related to software engineering using conventional to-do lists often leads to delays in project completion. Conventional project managers do not consider hard dependencies on task in relation to the rest of the project. There is an actual need for an intelligent terminal-based application that would be capable of mapping dependencies, determining the critical path of a project, and safely simulating “what-if” delays.

- **FUNCTIONAL REQUIREMENTS**

- **TASK MANAGEMENT:** The system must provide functionality to create, read, and remove tasks with properties such as name, duration, start date, and priority.
- **DEPENDENCY LINKING:** The system must provide functionality to link up tasks using prerequisite dependencies.
- **CYCLE DETECTION:** The system must prevent dependency links from forming any impossible cycles.
- **DATA SIMULATION:** The system must let the user enter a possible delay and visualize how this would affect the task.

- **CRITICAL PATH CALCULATION:** The system must calculate the longest path of dependent tasks determining the project deadline.
- **SMART STATUS NLP:** The system must understand natural conversation input (like "almost done", "halfway", etc.) and use it to change task status.
- **DATA PERSISTENCE:** The system must serialize and deserialize data in and out of a local .cnd file format.
- **NON-FUNCTIONAL REQUIREMENTS**

- **PERFORMANCE:** Graph traversal and sorting operations like topological sorting and critical path sorting should happen instantaneously within the CLI interface without any delay.
- **USABILITY:** The system must have user-friendly exception handling to catch exceptions such as **DateTimeParseException** and **NumberFormatException** and display error messages in a conversational manner.
- **PORTABILITY:** The entire application should be self-sufficient and work across all machines with Java Development Kit 14 or above versions without having any external dependencies.
- **DATA INTEGRITY:** The customized file parser must be able to gracefully handle missing fields and keep the DAG intact after loading.

- **SYSTEM ARCHITECTURE**

ChronoNexus utilizes a streamlined, layered architecture optimized for the terminal:

- **PRESENTATION LAYER (MAIN.java):** Manages standard I/O, CLI menu and NLP-lite text parser.
- **SERVICE / LOGIC LAYER (Scheduler.java):** Implements the algorithms, representing the project schedule as Directed Acyclic Graph (DAG) and conducting Kahn's Algorithm and DFS operations on it.
- **DOMAIN LAYER (Project.java, Task.java):** Specifies the domain entities and encapsulates the state.

- **PERSISTENCE LAYER (FileManager.java):** Implements the serialization of the Domain layer into a flat file-based text database (.cnd).

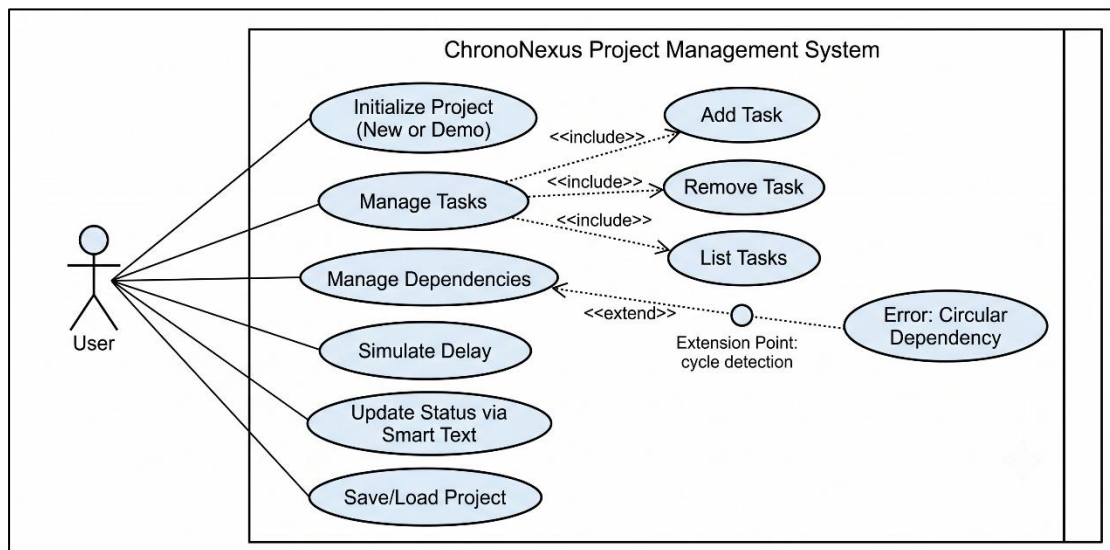
• **DESIGN DIAGRAMS**

• **USE CASE DIAGRAMS:**

- Actor: User

Use Cases:

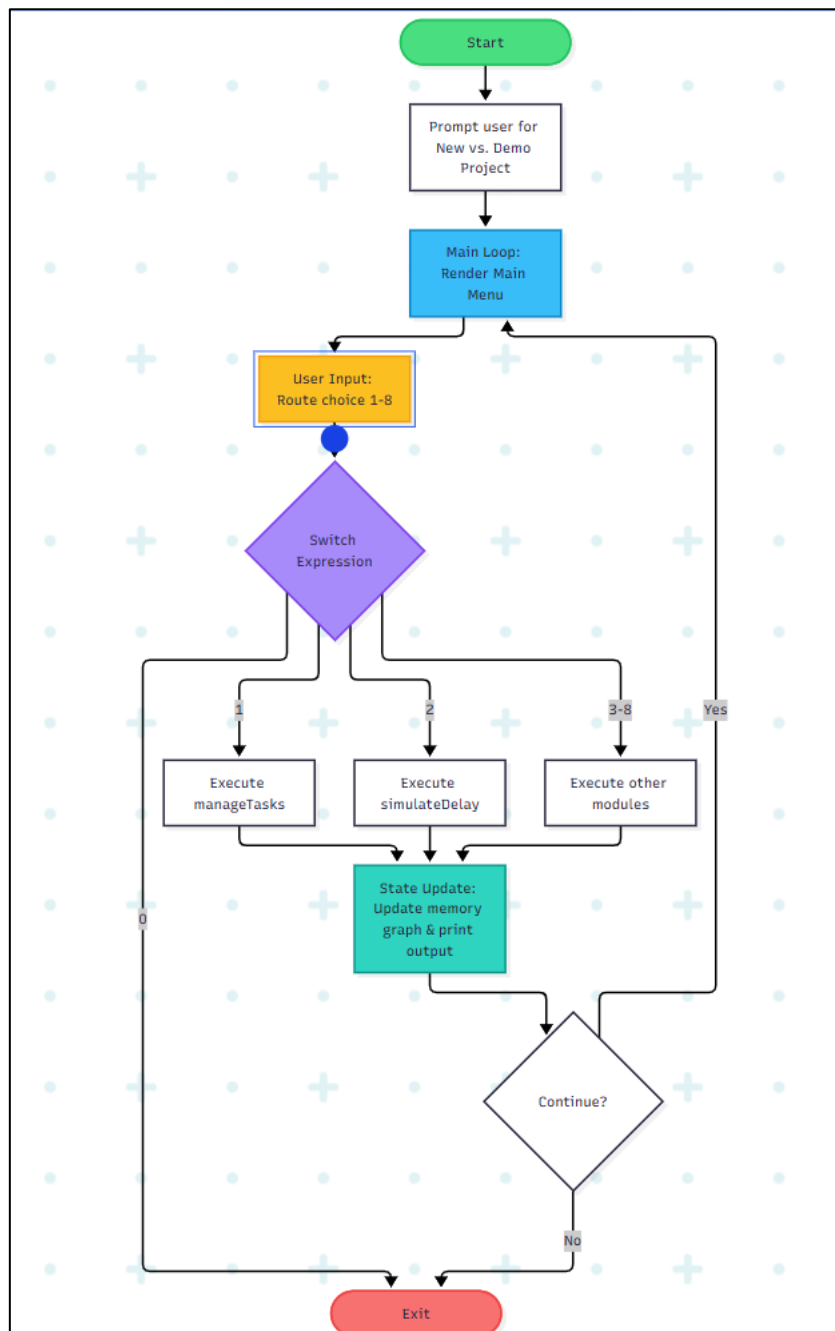
- Init Environment (New/Demo Project Load)
- Task Management (<<includes>> Task Addition, Deletion, Listing)
- Task Linking (<<extends>> Error Handling-Circular Dependency)
- Delay Simulation
- Task Progress Update (via Smart Text)
- Project Saving & Loading (.cnd file)



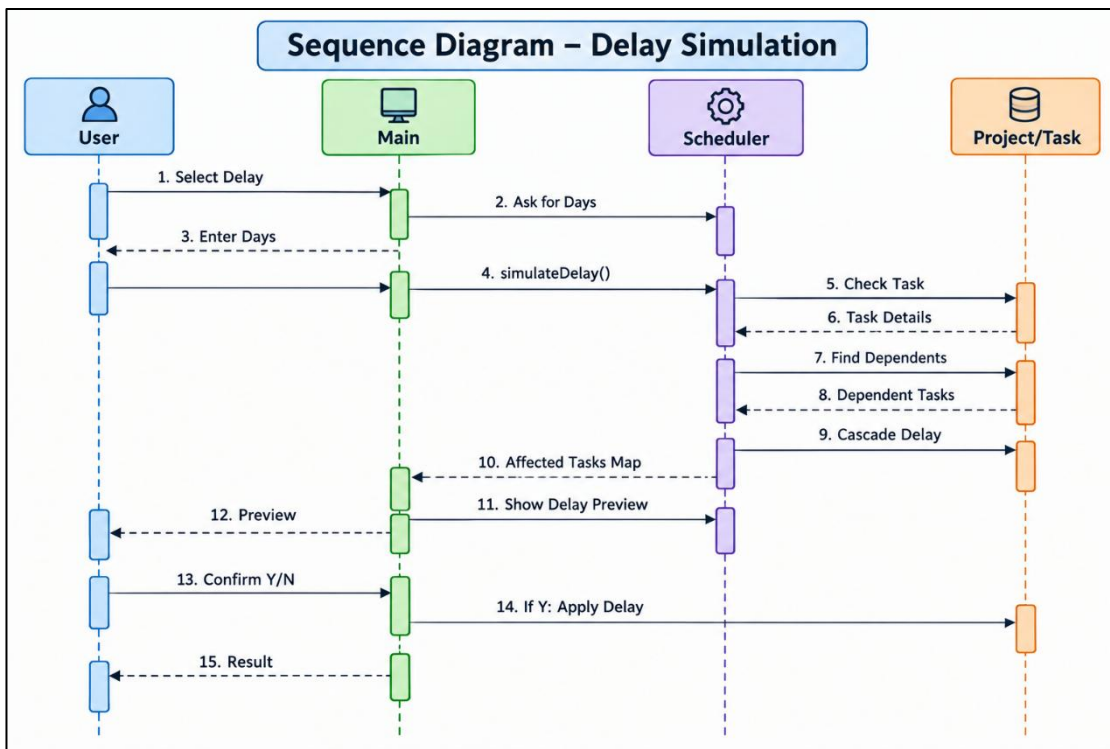
• **WORKFLOW DIAGRAM:**

- Start -> Request user input New/Demo project.
- Main Loop -> Draw main menu.

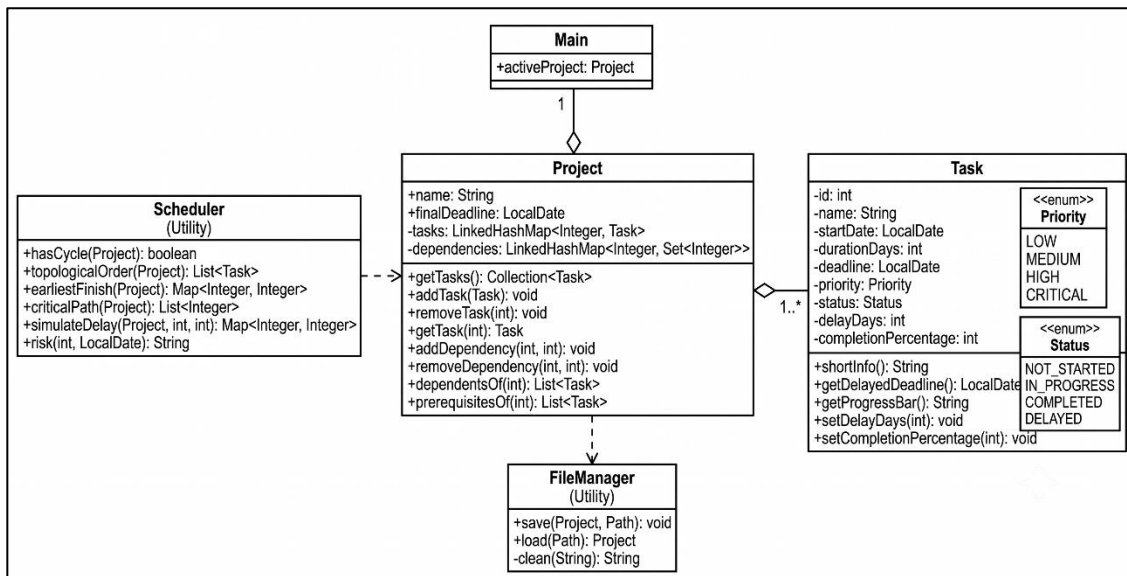
- User Choice -> Select route (1-8) using switch statement.
- Action Processed -> Call module function (such as manageTasks () or simulateDelay ()).
- Memory Update -> Change graph memory state / print results.
- Return -> Repeat main menu until Exit (0).



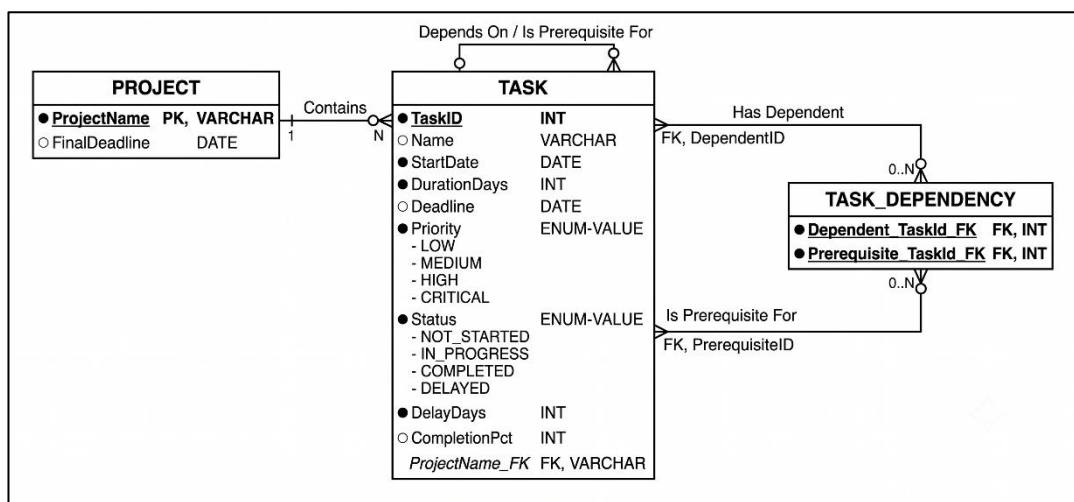
- **SEQUENCE DIAGRAM:**



• CLASS / COMPONENT DIAGRAM



• ER DIAGRAM



- **DESIGN DECISIONS AND RATIONALE**

- **DIRECTED ACYCLIC GRAPHS (DAGs):** Chosen over linear lists because real-world software and engineering projects operate in parallel. DAGs allow the system to mathematically guarantee that prerequisites are respected.
- **KHAN'S ALGORITHM:** Adopted to perform the topological sort because it efficiently finds isolated sub-graphs and naturally tracks indegrees, thus perfectly fitting into the resolution of task prerequisites.
- **CUSTOM FILE FORMAT (.cnd):** Rather than adopting JSON or XML files, decided to develop a custom pipe-delimited parser to keep the project entirely Core Java, providing ultimate portability without having to include additional frameworks such as Jackson or Gson.
- **NLP-LITE PARSER:** Rather than making users enter precise integers for their progress, a heuristic parser (parseSmartInput) has been developed to recognize words such as "almost," "half," and "wrap."

- **IMPLEMENTATION DETAILS**

The implementation leverages standard Java (JDK 14+) collections and time APIs.

- **Graph Structure:** The dependency adjacency list is mapped using a Map<Integer, Set<Integer>>. Integer prerequisiteID maps to a Set of downstream DependentIDs. LinkedHashSet is used to maintain deterministic insertion order for CLI rendering.
- **Enumerations:** Status (NOT_STARTED, IN_PROGRESS, COMPLETED, DELAYED) and Priority (LOW, MEDIUM, HIGH, CRITICAL) are enums, enforcing strict data typing throughout the scheduler.
- **Kahn's Algorithm:** Topological sorting (finding a logical sequence to execute parallel tasks respecting all dependencies) is implemented via Kahn's algorithm (indegree-based queue).
- **DFS Traversal:** Cycle detection is performed via Depth-First Search with recursion stack tracking (active nodes) to identify back-edges, dynamically preventing infinite dependency loops.

- **Gantt Representation:** A console-rendered progress bar [■■■■■-----] 50% is implemented algorithmically in `getProgressBar()`, providing basic Gantt chart visualization in a CLI environment.

- **SCREENSHOTS**

```
PS C:\Users\Dhruv\OneDrive\Desktop\ChronoApp> cd "c:\Users\Dhruv\OneDrive\Desktop\ChronoApp\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
=====
Welcome to ChronoNexus!
Master the Ripple, Conquer the Deadline.
=====
Would you like to start with the (1) Demo Project or a (2) New Project? [1/2]:
```

```
What would you like to do? 1

--- PROJECT OVERVIEW ---
It looks a bit empty here! There are no tasks in the project yet.

--- MAIN MENU ---
Current Project: JAVA Project | Target Deadline: 2026-10-16
1. View Project Overview & Progress
2. Manage Tasks (Add, List, or Remove)
3. Link Tasks (Set Dependencies)
4. Simulate a Delay (What-If Scenario)
5. View Critical Path (Longest chain of tasks)
6. Save Project
7. Load Project
8. Update Task Progress (Smart Text Input)
0. Exit
What would you like to do? █
```

```
What would you like to do? 2

--- MANAGE TASKS ---
1. List all Tasks
2. Add a new Task
3. Remove a Task
Choose an option: 2
What is the name of the task? Project Topic Discussion █
```

```
What would you like to do? 2
```

```
--- MANAGE TASKS ---
```

```
1. List all Tasks
```

```
2. Add a new Task
```

```
3. Remove a Task
```

```
Choose an option: 2
```

```
What is the name of the task? Project Topic Discussion
```

```
When does it start? (Format: YYYY-MM-DD): 2026-09-16
```

```
How many days will it take? 2
```

```
What is its priority? (LOW/MEDIUM/HIGH/CRITICAL): High
```

```
Perfect! 'Project Topic Discussion' was added successfully with ID: 1
```

```
What would you like to do? 8
```

```
--- PROJECT OVERVIEW ---
```

```
Overall Plan: 0% Complete (0 of 1 tasks finished)
```

```
-----  
ID: 1 | [-----] 0% | Project Topic Discussion | HIGH | NOT_STARTED | due 2026-09-17 (+0 delay)  
-----
```

```
Enter the ID of the task you want to update: 1
```

```
Current progress for 'Project Topic Discussion' is: 0%
```

```
How is it going? (e.g., 'just started', 'halfway', 'almost done', 'done', or a number like '75%'):
```

```
Your update: halfway
```

```
? Detected 50% completion! Task 'Project Topic Discussion' status is now: IN_PROGRESS
```

```
What would you like to do? 1
```

```
--- PROJECT OVERVIEW ---
```

```
Overall Plan: 0% Complete (0 of 1 tasks finished)
```

```
-----  
ID: 1 | [■■■■■-----] 50% | Project Topic Discussion | HIGH | IN_PROGRESS | due 2026-09-17 (+0 delay)  
-----
```

• TESTING APPROACH:

To ensure the reliability and mathematical accuracy of the scheduling engine, a multi-tiered testing strategy was employed, focusing heavily on graph logic validation and input resilience.

- **Algorithmic Verification (DAG Logic):** The core graph traversal algorithms were tested using isolated unit scenarios. `Scheduler.hasCycle()` was fed intentionally broken dependency loops (e.g., Task A → Task B → Task C → Task A) to verify that the Depth-First Search (DFS) instantly caught back-edges and rejected the operation before memory corruption occurred.

- **Real-World Scenario Integration Testing:** The system was validated against a practical, multi-phase software development timeline. The built-in "AI Viva Examiner" demo was used to trace a real workflow: establishing a Python 3.11 environment as a strict prerequisite before allowing the configuration of SpeechRecognition and PyAudio libraries and ultimately testing the voice-interactive grading logic. This confirmed that the topological sorting accurately respected strict sequential dependencies.
- **Simulation Sandbox Testing:** The recursive delay propagation was tested by applying a massive delay (e.g., 20 days) to an early prerequisite task. The output was manually verified to ensure that parallel tasks unaffected by the delay remained untouched, while directly dependent downstream tasks correctly reflected the cascaded delay.
- **NLP Parser Robustness:** The parseSmartInput () method was subjected to edge-case string inputs, including mixed alphanumeric phrases (e.g., "I am about 75% done" vs. "almost finished"), to ensure the heuristic keyword mapping correctly defaulted to safe integer values without throwing fatal exceptions.
- **Persistence I/O Testing:** The custom .cnd file format was tested by saving a complex project, manually modifying the delimited text file in an external editor to introduce formatting errors, and verifying that FileManager.load() handled the parsing safely.
- **CHALLENGES FACED**

Developing a CLI-based algorithmic scheduler entirely from scratch in standard Java presented several specific architectural challenges:

- **Dynamic Circular Dependency Prevention:** The most critical challenge was ensuring the Project DAG remained acyclic. If a user attempted to link a task that looped back on itself, saving that link would fatally break the topological sort. Implementing a temporary, predictive DFS traversal that checked the recursion stack for active nodes *before* committing the dependency to the adjacency list required careful state management.

- **Virtual Delay Cascading:** Simulating a delay required the system to calculate ripple effects without permanently altering the user's actual project data. Building an isolated virtual queue that recursively mapped extraDays to downstream tasks—while correctly ignoring parallel tasks that possessed enough buffer time to absorb the delay—was mathematically complex to implement without relying on third-party graph libraries.
- **Translating Natural Language to Quantitative Data:** Designing a command-line interface that felt modern and responsive was difficult within standard I/O constraints. Building an NLP-lite parser in pure Java to accurately map highly varied human inputs (such as "wrapping it up" or "just started") to strict status enums (IN_PROGRESS, COMPLETED) required an extensive, weighted keyword-matching heuristic.
- **Zero-Dependency Serialization:** Deciding to forgo heavy enterprise libraries like Jackson or Gson meant writing a custom flat-file database parser. Ensuring that the pipe-delimited (|) format could cleanly serialize and deserialize complex map structures—especially the graph's adjacency list—required meticulous string splitting and exception handling.

• LEARNING & KEY TAKEAWAYS:

Developing ChronoNexus provided profound insights into both applied graph theory and robust software engineering practices. Key takeaways include:

- **Bridging Theory and Practice:** Implementing Directed Acyclic Graphs (DAGs) demonstrated how abstract data structures solve highly practical problems. Using Depth-First Search (DFS) for cycle detection and Kahn's Algorithm for topological sorting proved essential for mathematically guaranteeing schedule viability.
- **Managing Real-World Complexity:** Applying the scheduling engine to intricate, multi-phase technical workflows—such as mapping out the strict prerequisites required to configure specific Python 3.11 environments and audio libraries for voice-interactive applications—highlighted why standard linear to-do lists are inadequate for engineering tasks.

- **Zero-Dependency Engineering:** Building the persistence layer from scratch using core Java `java.io` and `java.nio` packages reinforced the value of understanding foundational file serialization. Creating the custom `.cnd` parser eliminated reliance on heavy third-party libraries, resulting in a leaner, more portable application.
- **Heuristic Text Parsing:** Designing the "NLP-lite" smart status updater revealed the complexities of human-computer interaction in terminal environments. Mapping qualitative human phrases to quantitative data required careful consideration of keyword weighting and default fallbacks to prevent runtime crashes.

- **FUTURE ENHANCEMENTS**

While ChronoNexus successfully fulfills its current scope as a lightweight CLI tool, the architecture is designed to accommodate several future expansions:

- **Web-Based Graphical User Interface (GUI):** Transitioning the presentation layer from the command line to a modern, browser-based administrative dashboard. Utilizing HTML, CSS, and JavaScript would allow for highly interactive visual features, such as drag-and-drop dependency linking and dynamically rendered Gantt charts.
- **Multi-Project Portfolio Management:** Expanding the domain models to handle concurrent project schedules simultaneously. This would be highly beneficial for managing broader initiatives—such as launching distinct educational platforms, online schools, or coaching hubs—where resources are shared across multiple independent timelines.
- **Machine Learning Integration:** Upgrading the current heuristic keyword parser to a lightweight machine learning model to better understand contextual natural language inputs and colloquialisms for status updates.
- **Export Capabilities:** Adding utility methods to the `FileManager` class to export the calculated critical path and topological order into standardized `.csv` formats for easy integration with spreadsheet software.

• **REFERENCES**

- Oracle Corporation. (2020). *Java Platform, Standard Edition & Java Development Kit Version 14 API Specification*. Retrieved from docs.oracle.com.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. (Reference for Depth-First Search and Directed Acyclic Graph traversal algorithms).
- Kahn, A. B. (1962). "Topological sorting of large networks." *Communications of the ACM*, 5(11), 558–562. (Reference for the fundamental logic used in the topological order scheduler).
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (Reference for the layered software architecture and separation of concerns implemented in the codebase).